

EPIC 08 — Task 04: Testing Microservices

The 5 types of tests:

1. Unit tests
2. Integration tests
3. API tests
4. Service communication tests
5. Failure tests

Below is the implementation you can use in your **Spring Boot + Java 17** project.

Step 1 — Add testing dependencies

Open your pom.xml.

Add:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-test</artifactId>
  <scope>test</scope>
</dependency>
```

This provides JUnit 5, Mockito, MockMvc, assertions, etc.

Save the file and run:

```
mvn clean test
```

Step 2 — Unit Test

A unit test tests one class independently.

Suppose you have:

```
src/main/java
└─ com.example.orderservice
    └─ service
        └─ OrderService.java
```

Example service:

```
@Service
public class OrderService {

    public String createOrder() {
        return "Order created successfully";
    }
}
```

Create:

```
src/test/java
└─ com.example.orderservice
    └─ service
        └─ OrderServiceTest.java
```

Add:

```
package com.example.orderservice.service;

import static org.junit.jupiter.api.Assertions.assertEquals;

import org.junit.jupiter.api.Test;

class OrderServiceTest {

    private final OrderService orderService = new OrderService();

    @Test
    void shouldCreateOrder() {

        String result = orderService.createOrder();
```

```
        assertEquals("Order created successfully", result);
    }
}
```

This is your **Unit Test**.

Step 3 — Integration Test

Integration testing verifies that multiple parts of your application work together.

Create:

```
src/test/java/com/example/orderservice/OrderServiceIntegrationTest.java
```

Add:

```
package com.example.orderservice;

import static org.junit.jupiter.api.Assertions.assertNotNull;

import org.junit.jupiter.api.Test;
import org.springframework.boot.test.context.SpringBootTest;

@SpringBootTest
class OrderServiceIntegrationTest {

    @Test
    void applicationContextLoads() {

        assertNotNull(OrderServiceIntegrationTest.class);
    }
}
```

The important part is:

```
@SpringBootTest
```

It starts the Spring application context and checks that your application components can be loaded together.

Step 4 — API Test

For API testing, use `MockMvc`.

Create:

```
src/test/java/com/example/orderservice/controller/OrderControllerTest.java
```

Example:

```
package com.example.orderservice.controller;

import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static
org.springframework.test.web.servlet.result.MockMvcResultMatchers.status;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.test.web.servlet.MockMvc;

@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Test
    void shouldReturnOrders() throws Exception {

        mockMvc.perform(get("/orders"))
            .andExpect(status().isOk());
    }
}
```

Now the test calls:

GET /orders

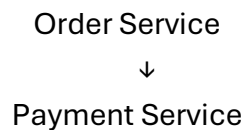
and verifies that the API returns:

HTTP 200 OK

Change /orders to your actual API endpoint if your controller uses a different path.

Step 5 — Test Service Communication

Suppose:



The Order Service calls the Payment Service.

You should test what happens when the Payment Service responds successfully.

For example, using Mockito:

```
@Mock
private PaymentClient paymentClient;
```

```
@InjectMocks
private OrderService orderService;
```

Example test:

```
@Test
void shouldCommunicateWithPaymentService() {

    when(paymentClient.processPayment())
        .thenReturn("PAYMENT_SUCCESS");

    String result = orderService.processOrder();
```

```
assertEquals("PAYMENT_SUCCESS", result);

verify(paymentClient).processPayment();
}
```

This verifies that:

- Order Service calls Payment Service.
- Payment Service returns the expected response.
- Order Service handles that response.

Step 6 — Failure Test

Now test what happens when another service is unavailable.

Example:

```
@Test
void shouldHandlePaymentServiceFailure() {

    when(paymentClient.processPayment())
        .thenReturn(new RuntimeException("Payment service unavailable"));

    assertThrows(
        RuntimeException.class,
        () -> orderService.processOrder()
    );
}
```

This tests the failure scenario:

Order Service
↓
Payment Service
X
Service unavailable

Your application should not silently ignore this failure.

Step 7 — Test API Failure

You should also test invalid requests.

For example:

```
@Test
void shouldReturnNotFoundForInvalidOrder() throws Exception {

    mockMvc.perform(get("/orders/99999"))
        .andExpect(status().isNotFound());
}
```

This verifies that an invalid/non-existent order produces:

404 NOT FOUND

If your application currently returns to another status, use the status defined by your actual controller.

Step 8 — Run All Tests

In Eclipse:

Right-click your project → Run As → Maven test

Or from the terminal:

```
mvn test
```

You should see:

Tests run: ...

Failures: 0

Errors: 0

Skipped: 0

Step 9 — Recommended test structure

Your project should now look like:

```
src
├── main
│   └── java
│       └── com.example.orderservice
│           ├── controller
│           ├── service
│           ├── repository
│           └── ...
└── test
    ├── java
    │   └── com.example.orderservice
    │       ├── service
    │       │   └── OrderServiceTest.java
    │       ├── controller
    │       │   └── OrderControllerTest.java
    │       └── OrderServiceIntegrationTest.java
```

Test

Unit test

What you test

Individual service/business logic

Integration test

Spring application/components together

API test

REST endpoints and HTTP responses

Service communication test

Communication between microservices

Failure test

Service/API failure and error handling

Finally run:

`mvn clean test`

If all tests pass, **Task 04 — Testing Microservices is complete.**